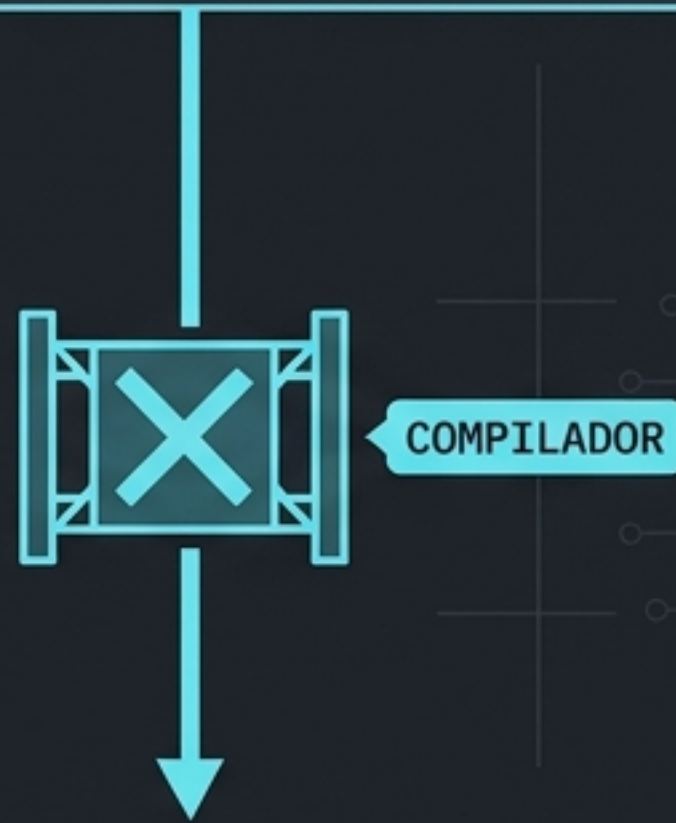


Arquitectura de Software a Prueba de Fallos

Excepciones y Manejo de Ficheros en Java

Fuente: Tema 13

Errores de Compilación



El compilador **avisa**; el programa **no se ejecuta** hasta solucionarlo (ej. falta un punto y coma).

Errores en Tiempo de Ejecución (Excepciones)



Ocurren durante la ejecución. Provocan un **cierre abrupto**, **pérdida de datos** y fuerzan al usuario a **reiniciar**.

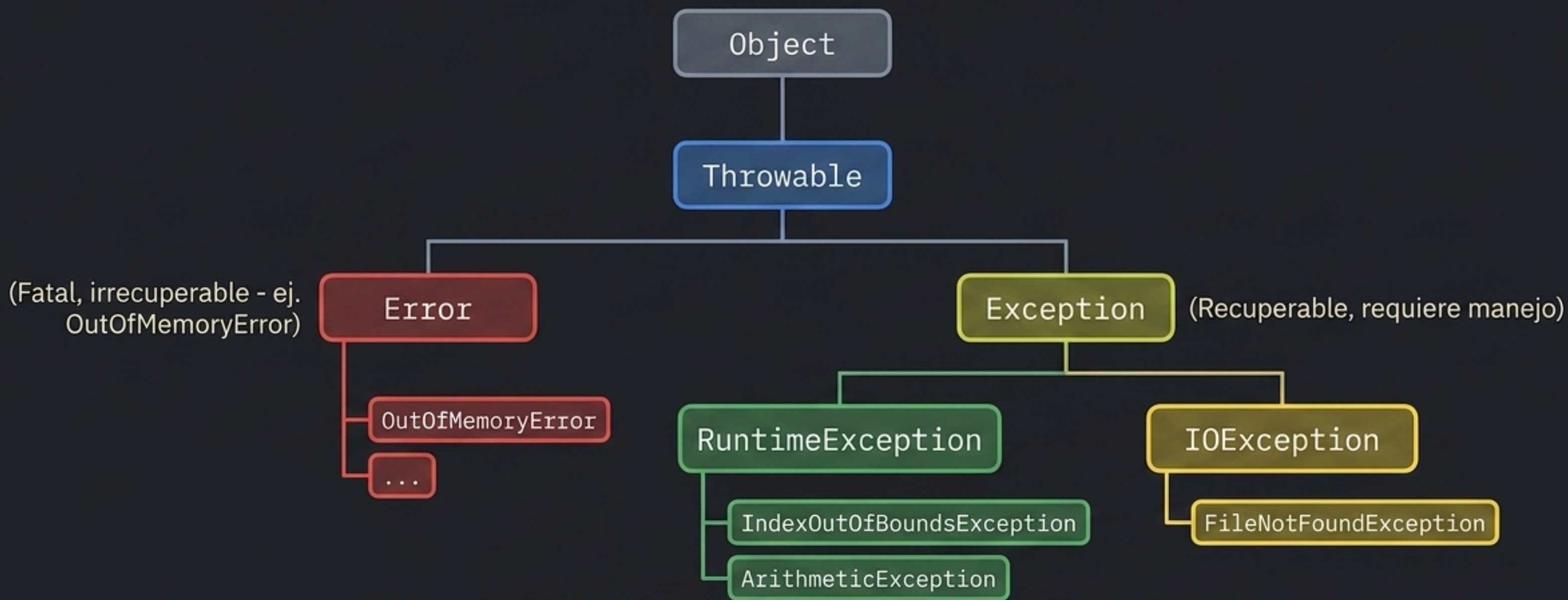
// Villanos Comunes en Tiempo de Ejecución

✂ `InputMismatchException`

📊 `ArithmeticException` (división entre cero)

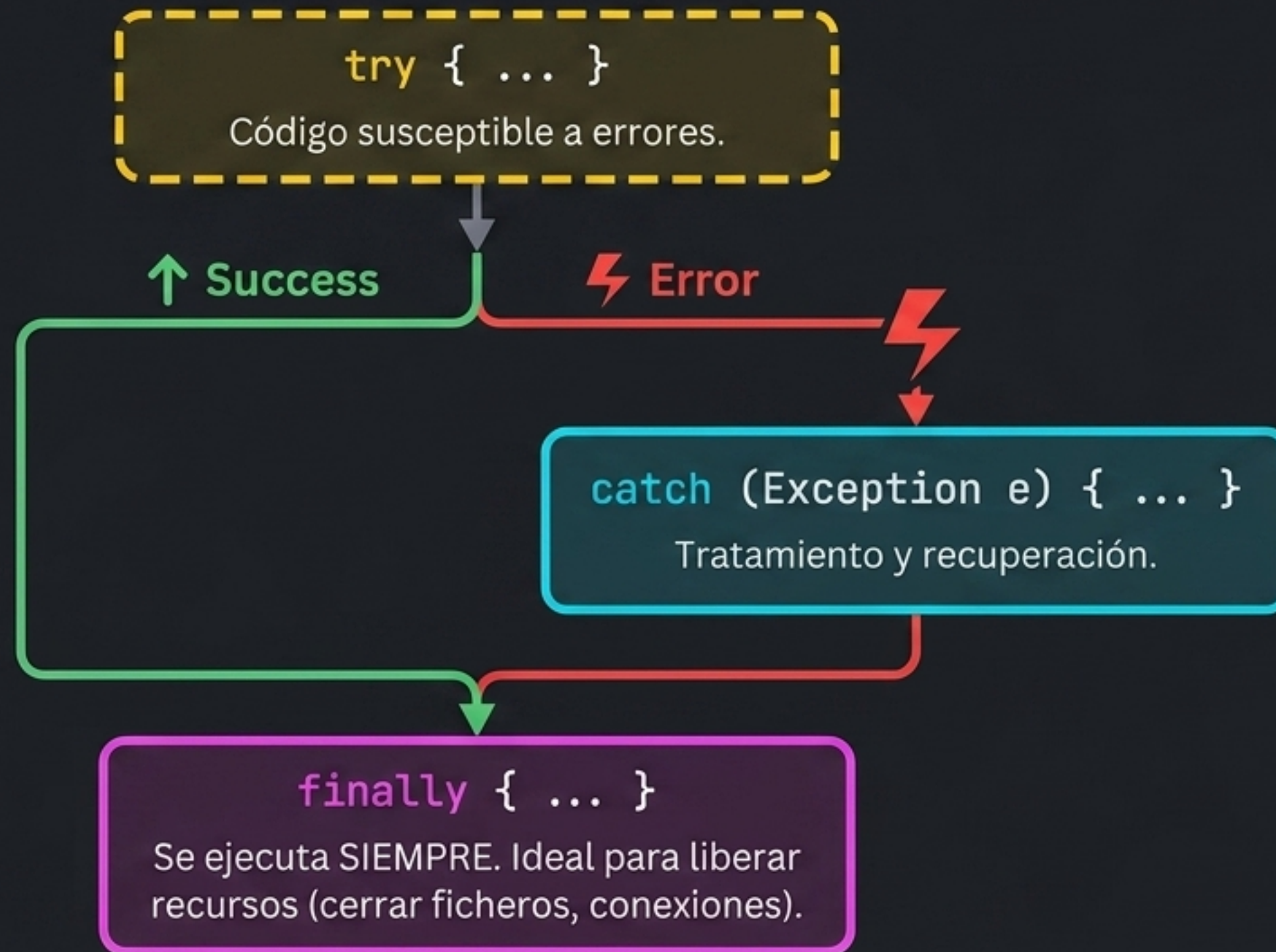
📄 `IndexOutOfBoundsException`

Java Exception Taxonomy



Todas las excepciones manejables en Java heredan de la clase base Exception.

Manejo de Excepciones: Try-Catch-Finally Flow



✓ El usuario no percibe el fallo abrupto; el programa se recupera de forma exitosa.

Escalera del Infierno

```
static int Main(string[] args)
{
    try {
        try {
            try {
                try {
                    try {
                        try {
                            try {
                                try {
                                    Console.WriteLine("Hello World");
                                } catch (Exception) {}
                            } catch (Exception) {}
                        } catch (Exception) {}
                    } catch (Exception) {}
                } catch (Exception) {}
            } catch (Exception) {}
        } catch (Exception) {}
    } catch (Exception) {}
    return 0;
}
```

Redundancia y código inflado.

Multicatch Elegante

```
public static void main(String[] args) {
    double a = 4, b = 5;

    try {
        double resultado = a / b;
        System.out.println("Resultado: " + resultado);
    } catch (NullPointerException | ArithmeticException ex) {
        System.out.println("ERROR: " + ex.getMessage());
    }
}
```

Agrupar excepciones que requieren el mismo tratamiento en un solo bloque, mejorando la legibilidad y el mantenimiento.

ORIGINAR (throw)



```
throw new Exception("La persona ha de  
ser mayor de edad");
```

Se utiliza dentro del bloque de código para instanciar y lanzar activamente una excepción (ej. validaciones de lógica de negocio).

PROPAGAR (throws)



```
public static void metodoA() throws  
Exception
```

Se coloca en la firma del método. Delega la responsabilidad de capturar el error al bloque de código que invocó este método.

Implementar constructor con parámetro String.

2

```
class DNIException extends Exception {  
    public DNIException(String mensaje) {  
        super(mensaje);  
    }  
}
```

1
Regla de oro: Debe heredar de Exception (estándar de nomenclatura terminando en "Exception").

3

Pasa el mensaje descriptivo a la clase padre para cuando se consulte el motivo del error.

Reglas Profesionales

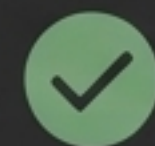
```
/**
 * Divide dos números enteros.
 *
 * @param numerador el numerador de la división
 * @param denominador el denominador de la división
 * @return el resultado de la división
 * @throws ArithmeticException si el denominador es cero
 */
public int dividir(int numerador, int denominador)
    throws ArithmeticException {
    if (denominador == 0) {
        throw new ArithmeticException("División por cero");
    }
    return numerador / denominador;
}
```



Usar la etiqueta `@throws` por cada excepción lanzada.



Describir exactamente la `condición` bajo la cual se lanza.



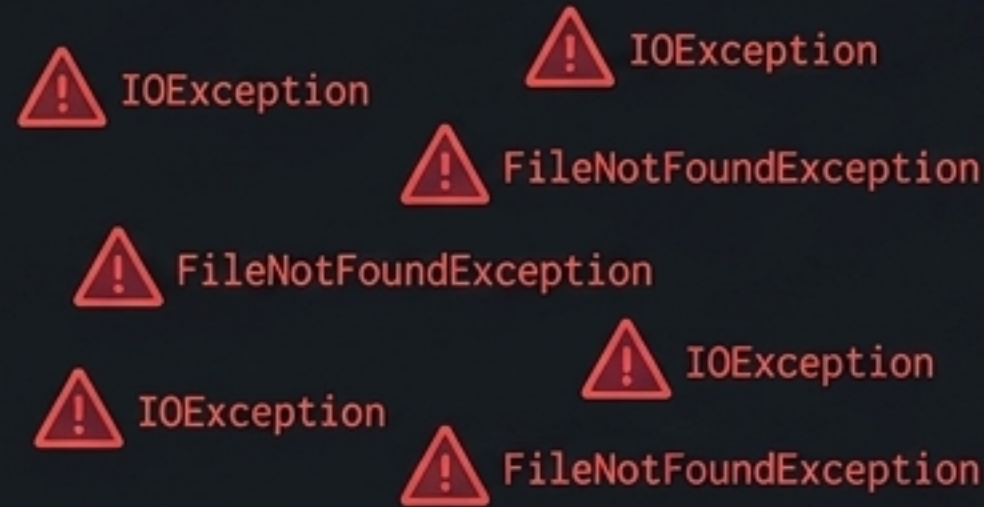
Facilita que otros desarrolladores sepan qué deben capturar.

Memoria del Programa (JVM)

Variables, arrays, control flow.

Sistema de Archivos del SO

Discos, ficheros, permisos.



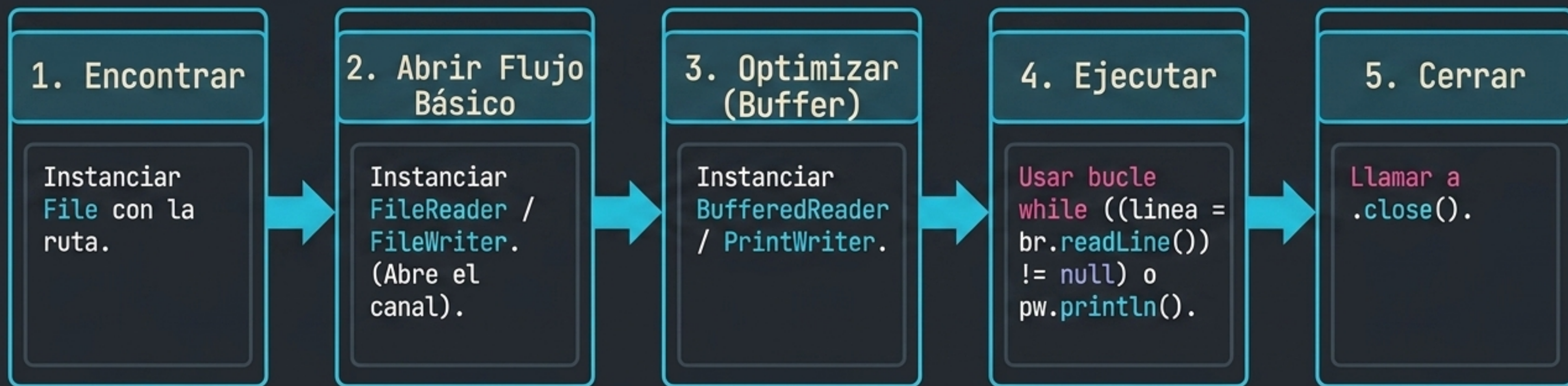
El Campo de Batalla: Ficheros. Interactuar con el exterior es impredecible (archivos borrados, discos corruptos, falta de permisos). Por eso, el manejo de ficheros y las excepciones son conceptos inseparables en Java.

File

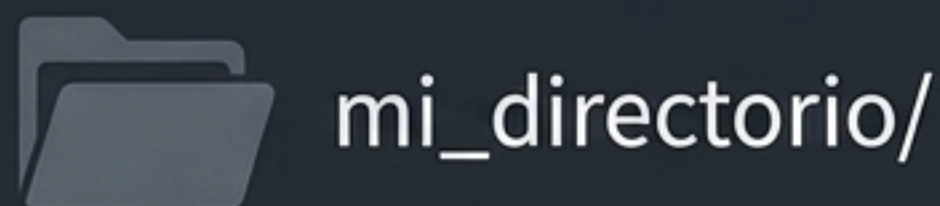
(Para abrir/localizar un fichero en la ruta)

	LECTURA (Entrada)	ESCRITURA (Salida)
Carácter a Carácter	FileReader	FileWriter
Línea a Línea (Optimizado)	BufferedReader	PrintWriter

Las clases de línea a línea envuelven a las de carácter a carácter para proporcionar métodos como `.readLine()` o `.println()`.



Crítico: Cerrar desde el objeto más externo al más interno para liberar memoria.



`mkdir()` o `mkdirs()`

(crea el directorio o los padres si no existen).



`listFiles()`

(devuelve un array `File[]` con los contenidos).



`delete()`

(elimina el directorio).

Todo esto se gestiona mediante la misma clase fundamental: `java.io.File`.

Tradicional (Legacy)

`java.io.File`

Operaciones básicas y directas.

Métodos devuelven boolean en caso de error (menos detalle).

```
archivo.createNewFile();  
archivo.delete();
```

VS

Moderno NIO.2 (Java 7+)

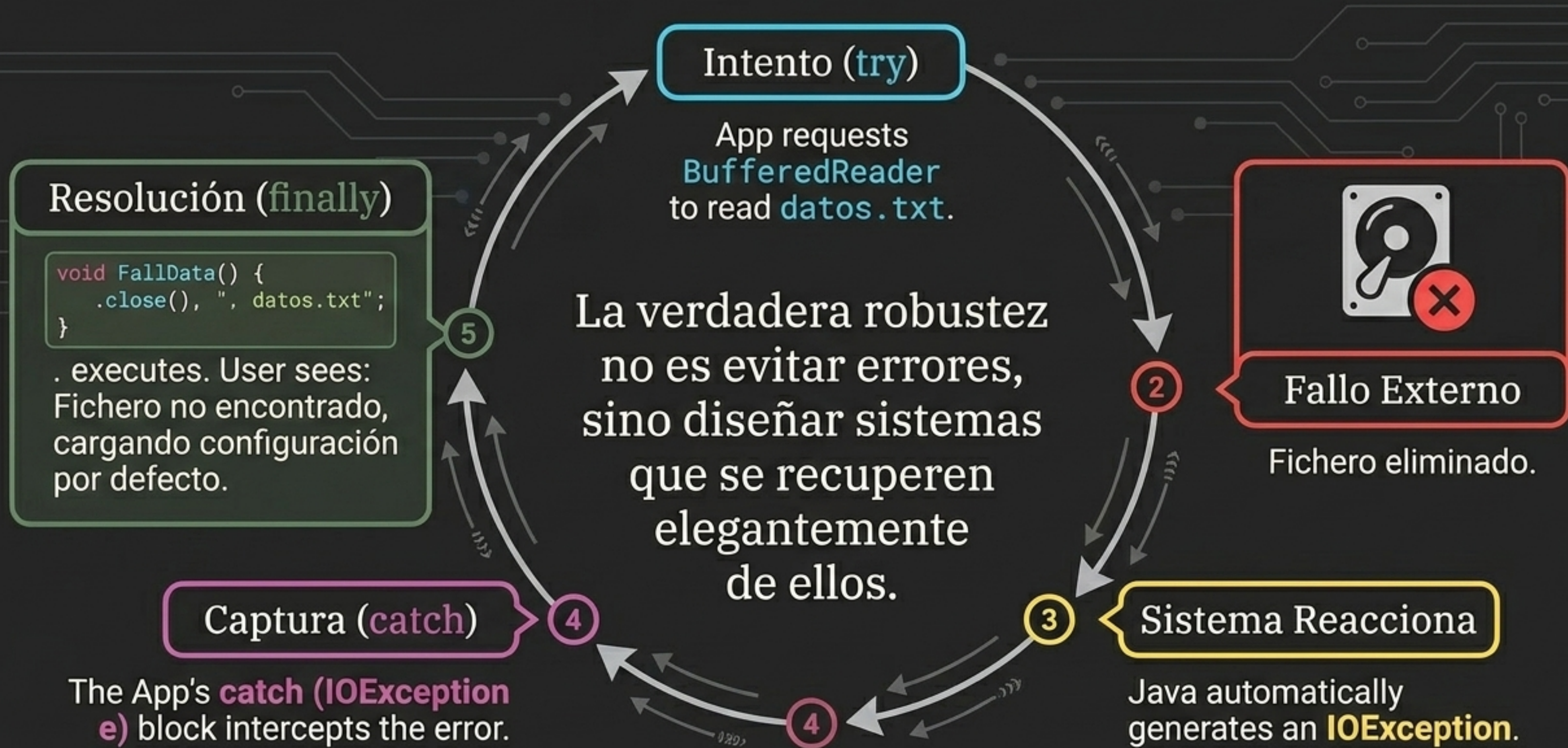
`java.nio.file.Path & Files`

Manejo más robusto y flexible del sistema de archivos.

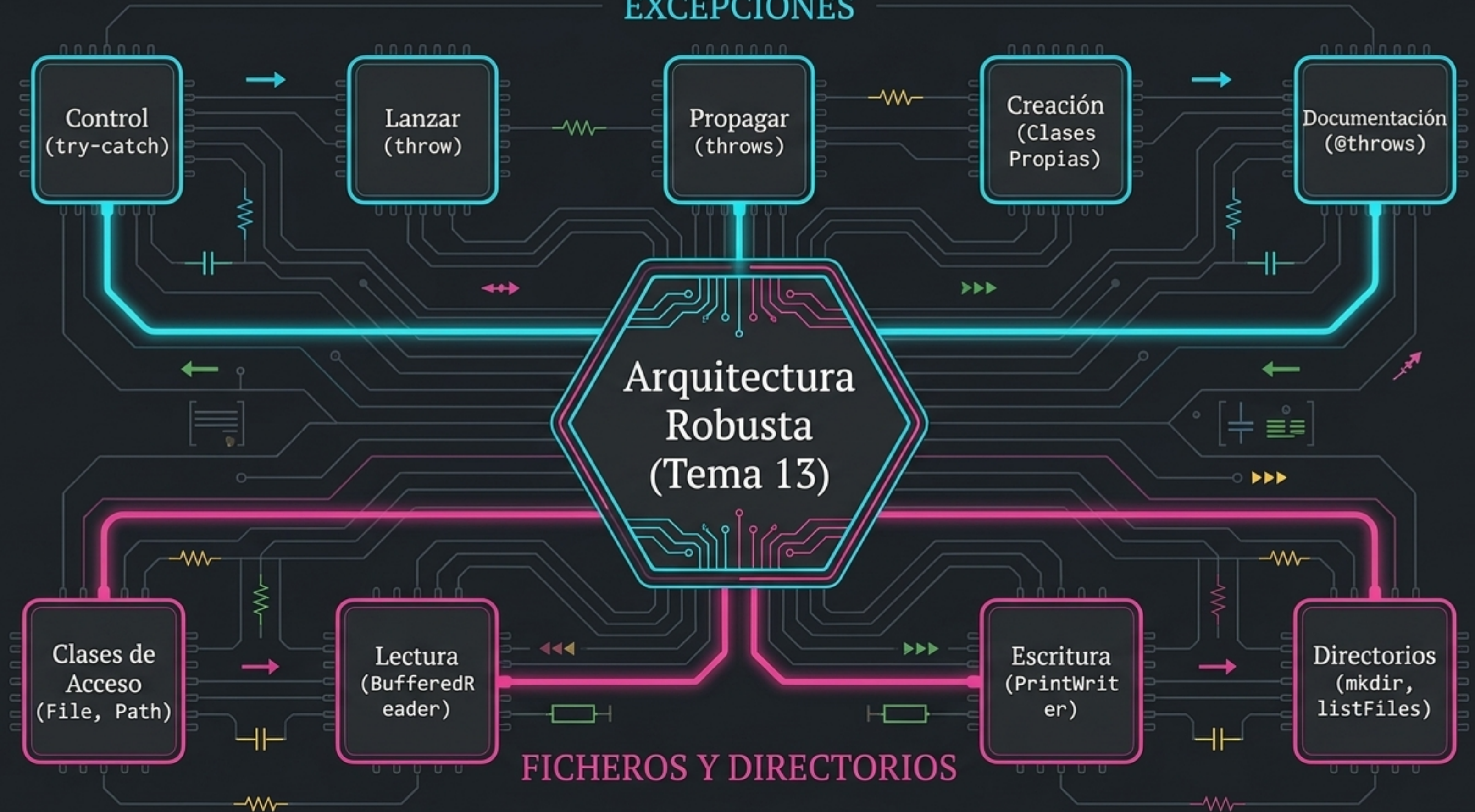
Lanza excepciones detalladas y específicas en caso de fallo.

```
Files.createFile(archivoPath);  
Files.deleteIfExists(archivoPath);
```

Flujo de Recuperación Elegante de Errores



EXCEPCIONES



Del error impredecible a la ejecución controlada.